

## FLEXBMS

# System Overview

Home battery energy storage system

---

|                           |                                 |
|---------------------------|---------------------------------|
| <b>Document date</b>      | 30 July 2026                    |
| <b>System</b>             | 96s LiFePO4 home BESS           |
| <b>Nominal capacity</b>   | 314 Ah / approximately 96.5 kWh |
| <b>Primary controller</b> | STM32G491                       |
| <b>Document status</b>    | Living architecture overview    |

This document explains the current FlexBMS system at architecture level. Schematics and source code remain authoritative for implementation details.

# Contents

---

|                                        |   |
|----------------------------------------|---|
| 1. System level .....                  | 3 |
| 1.1. System roles .....                | 3 |
| 1.2. Energy and information flow ..... | 3 |
| 2. Hardware .....                      | 3 |
| 2.1. Battery assembly .....            | 3 |
| 2.2. Module boards .....               | 4 |
| 2.3. Master controller .....           | 4 |
| 2.4. High-voltage path .....           | 4 |
| 2.5. Physical communications .....     | 5 |
| 3. Software .....                      | 5 |
| 3.1. STM32 .....                       | 5 |
| 3.2. HV supervisor .....               | 6 |
| 3.3. ESP32 .....                       | 6 |
| 3.4. Energy management system .....    | 6 |
| 3.5. Home Assistant integration .....  | 7 |
| 3.6. Companion application .....       | 7 |
| 4. Operating concept .....             | 7 |
| 4.1. Start-up .....                    | 7 |
| 4.2. Normal operation .....            | 7 |
| 4.3. Faulted operation .....           | 7 |
| 5. Interfaces and ownership .....      | 8 |
| 6. Current project structure .....     | 8 |
| 7. Open topics .....                   | 8 |

# 1. System level

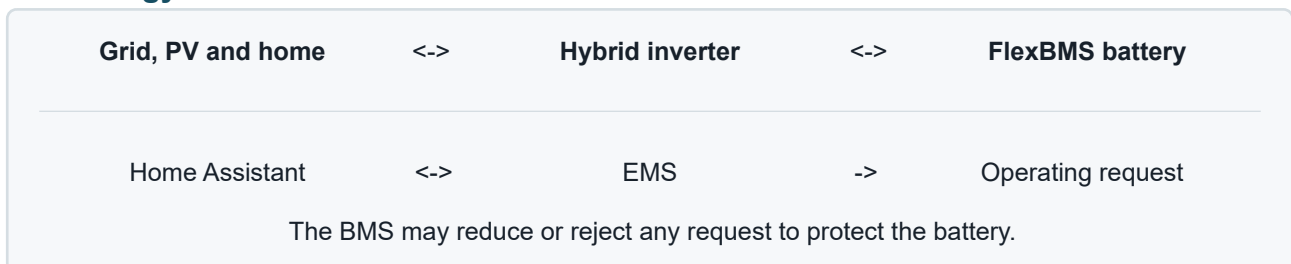
FlexBMS is the battery management and integration platform for a stationary home battery energy storage system. The battery consists of 96 LiFePO<sub>4</sub> cells in series, arranged as four 24-cell modules. At a nominal cell voltage of 3.2 V and a capacity of 314 Ah, the pack is approximately 307 V and 96.5 kWh.

The system connects the battery to a three-phase hybrid inverter. FlexBMS measures the battery, protects it, controls the high-voltage connection, and provides the information needed by higher-level energy management. The battery management system remains the final authority for battery safety.

## 1.1. System roles

| Layer                    | Responsibility                                                                             | Status  |
|--------------------------|--------------------------------------------------------------------------------------------|---------|
| Battery and HV hardware  | Stores energy, measures physical quantities, and safely connects the pack to the inverter. | CURRENT |
| STM32 BMS                | Performs real-time measurement, protection, state handling, and contactor control.         | CURRENT |
| ESP32 gateway            | Publishes telemetry to the local network while remaining listen-only on CAN.               | PLANNED |
| Energy management system | Chooses when and how much to charge or discharge within BMS limits.                        | PLANNED |
| Home Assistant           | Provides dashboards, automation, and a user-facing view of the installation.               | PLANNED |

## 1.2. Energy and information flow



### Control principle

Energy management is advisory; battery protection is mandatory. The EMS may request an operating point, but the BMS defines the permitted charge and discharge envelope and can open the contactor.

# 2. Hardware

## 2.1. Battery assembly

The pack contains 96 series-connected LiFePO<sub>4</sub> cells divided into four physical 24s modules. Each module contains two twelve-cell monitoring sections. A 200 A / 75 mV shunt is located at the negative end of the pack for current measurement.

| Item                  | Current baseline       |
|-----------------------|------------------------|
| Cell chemistry        | LiFePO4                |
| Series configuration  | 96s                    |
| Physical modules      | 4 x 24s                |
| Cell capacity         | 314 Ah                 |
| Nominal pack voltage  | 307.2 V                |
| Nominal stored energy | Approximately 96.5 kWh |
| Current shunt         | 200 A / 75 mV          |

The modules are installed in an unheated garage. Temperature measurement is distributed across the pack. The current STM32 configuration expects four NTC inputs per battery-monitor IC. Final sensor population and harness documentation should be kept consistent with the assembled modules.

## 2.2. Module boards

Each 24s module has one FlexBMS module board with two MC33771C battery cell controllers. Each controller measures twelve cells, supports passive balancing, reads local temperature sensors, and participates in the trans-former-isolated TPL daisy chain.

Across the complete pack this gives:

- eight MC33771C devices;
- 96 monitored cell voltages;
- up to 32 configured NTC measurements; and
- one current-sensing location on the first, lowest-voltage controller.

## 2.3. Master controller

The master board is the central interface between the battery, high-voltage switching hardware, service tools, inverter side, and local network.

Its main functional blocks are:

- STM32G491 real-time controller;
- MC33664 TPL transceiver for the module-board chain;
- ESP32-C3-WROOM-02U connectivity controller;
- isolated CAN and RS485 interfaces;
- isolated measurements of battery-side and load-side high voltage;
- contactor and precharge relay drivers;
- USB for flashing, diagnostics, and the Companion application; and
- power selection between USB and the battery-derived low-voltage supply.

The board contains separate HV, LV, and inverter-side isolated domains. USB-only power supports development and diagnostics, but does not provide the 12 V rail required to energize the contactor.

## 2.4. High-voltage path

The main contactor is located in the positive battery path. A separate precharge path limits inrush current before the contactor closes. The STM32 is the sole software owner of both drivers. It measures the battery and load sides through separate isolated AMC3330 channels, compares the battery-side result with the voltage reported by the cell-monitor chain, and observes both active-low diagnostic outputs.

The contactor is pulled in at full drive and then held with 20 kHz PWM. Precharge remains closed during pull-in and opens only after the contactor drive is established. The hold duty is a firmware configuration value.

Voltage feedback shows whether the two HV nodes are plausible and whether precharge has progressed. It does not prove the mechanical position of the contactor: the load-side voltage may remain present after opening because the inverter DC-link discharge behavior is not yet characterized.

Pack-level protection also includes the main DC fuse and a manual DC isolation device. Exact component ratings, wiring, creepage, clearance, and protection coordination belong in the hardware design files and commissioning documentation.

## 2.5. Physical communications

| Interface | Connection                       | Purpose                                          |
|-----------|----------------------------------|--------------------------------------------------|
| TPL       | Master to 8 monitor ICs          | Cell, temperature, diagnostic, and current data. |
| USB       | Service computer to STM32        | Firmware, logs, and Companion access.            |
| CAN       | STM32 and inverter-side domain   | Battery status and inverter communication.       |
| Wi-Fi     | ESP32 to local network           | Telemetry, local maintenance page, and MQTT.     |
| RS485     | ESP32 to inverter-side equipment | Fallback or supplementary Modbus connection.     |

## 3. Software

The software is divided by responsibility. Time-critical protection and actuation remain on the STM32. Network-facing functions belong on the ESP32. Site optimization belongs in the EMS, with Home Assistant providing the local user and automation layer.

### 3.1. STM32

**CURRENT**

The STM32 firmware is the real-time BMS. It starts and monitors the MC33771C chain, gathers a complete battery measurement set, evaluates battery and communication faults, and controls relay permission.

The current battery profile is:

- eight MC33771C devices with twelve cells each;
- four NTC channels per controller on GPIO0 through GPIO3;
- current sensing on CID 1 with a 375 micro-ohm shunt value;
- positive current for charging and negative current for discharging;
- 314 Ah nominal capacity; and
- amp-hour retention in RTC backup registers while backup power is present.

The firmware separates active fault conditions, acknowledgement-required latches, and in-memory fault history. The HV supervisor receives permission only while the BMS is running without blocking faults and holds one fresh, complete measurement set from the monitor chain.

Runtime faults keep measurement and monitoring active while preventing relay closure. A clear request is accepted only after the underlying runtime condition has disappeared. The BMS then repeats its device, register, diagnostic, and measurement validation before allowing the latch to clear.

The STM32 also hosts:

- the dedicated HV supervisor;
- board-level analog and digital I/O;
- CAN communication;
- USB serial communication;
- and the browser-based Companion service interface.

#### 3.1.1. Todo:

High priority:

- Go through CubeMX configuration. Pinout is OK.
- Watchdog implementation. Contactor should open in case of timeout.
- Analog measurement of HV side. Including diagnostic pin checking. ADC configuration, high accuracy low Fs.
- Precharge handling. Including:
- Contactor handling. Including: economizing, failure checking using voltages (to be discussed)
- USB VSENSE setup
- Derating on temperature (mainly low temperature charging, high temperature is not that applicable)
- Communication to Goodwe inverter. Exact protocol needs discussion and likely imperial data once we have the inverter ready.
- User LED

Medium priority:

- Communication to ESP32 over UART. Reporting and named service operations; see [Documentation/architecture/home-bess-firmware-and-maintenance.md](#).
- Aux relay control (potentially useful for a future heating system)

## 3.2. HV supervisor

### CURRENT

The HV supervisor follows OFF -> SELF\_TEST -> PRECHARGE -> CONTACTOR\_CLOSE -> RUN. The planned transport-independent `setRunRequest(bool)` service operation provides intent only; it does not command a contactor. The STM32 accepts operation only when its BMS and HV-supervisor conditions are healthy. Until this interface is implemented, the supervisor remains OFF by default.

SELF\_TEST checks BMS permission, measurement freshness, isolated-voltage diagnostics, agreement between the battery-side and cell-monitor voltages, an unenergized load side, and the available power source. PRECHARGE raises the load-side voltage and requires a stable voltage relationship before contactor pull-in. CONTACTOR\_CLOSE transfers from the precharge path to contactor hold drive. RUN keeps monitoring the BMS permission and both HV measurements.

A removed request, lost BMS permission, or implausible HV feedback disables both drivers. A blocking fault does not erase a true run request: after an explicit user fault-clear request and successful STM32 revalidation, the BMS may resume the startup sequence. The request defaults off after STM32 reset or loss of the requesting service transport. The STM32 reports requested state, actual HV state, and any blocking reason separately.

USB-only operation supports service access but cannot request or energize the HV path. The fitted hardware pull-down defines the inactive USB-sense level; the STM32 does not enable an internal pull.

### Source of truth

Thresholds, timings, fault definitions, CAN identifiers, and register-level behavior are intentionally not duplicated here. The STM32 source code and hardware schematics are authoritative.

## 3.3. ESP32

### PLANNED

The ESP32 is the network and telemetry controller in the isolated inverter-side domain. Its intended role is to observe BMS and inverter information, publish selected data to the local network, and support integration with the EMS and Home Assistant.

The ESP32 is not part of the battery safety chain. It is not allowed to transmit commands on the BMS CAN bus and must remain listen-only there. Loss of the ESP32 or Wi-Fi must not prevent the STM32 from protecting or isolating the battery.

No ESP32 firmware is currently present in this repository. The agreed target is a local telemetry and maintenance Gateway: it uses isolated UART to the STM32, serves a BMS-only Companion page locally, publishes to Home Assistant through MQTT, and remains listen-only on shared BMS/GoodWe CAN. It is not a safety authority. The detailed transport, update, local-network, and recovery design is in [Documentation/architecture/home-bess-firmware-and-maintenance.md](#).

### 3.3.1. Pinout

Logical pin numbering on ESP32-C3-WROOM-02U-N4:

1. 3V3
2. EN
3. IO4 - CAN\_TX
4. IO5 - CAN\_RX

To be continued...

## 3.4. Energy management system

### PLANNED

The EMS is the supervisory optimization layer. It should combine battery availability with site demand, PV production, inverter limits, dynamic electricity prices, EV charging, and other flexible loads.

The EMS may request charging or discharging, but it must operate inside limits supplied by the BMS and inverter. It must tolerate missing telemetry and fall back to a safe, predictable operating policy.

The EMS implementation and deployment location are not yet defined in this repository. Its first interface should be a small, explicit contract containing battery state, available charge/discharge power, requested power, data freshness, and reason/status information.

### 3.5. Home Assistant integration

**PLANNED**

Home Assistant is the intended user-facing integration point for the installation. It can provide:

- battery, inverter, PV, grid, and load dashboards;
- fault and availability notifications;
- manual operating-mode selection;
- automation around dynamic prices and flexible loads; and
- long-term energy and temperature history.

Home Assistant must not become a safety dependency. If Home Assistant is unavailable, the STM32 must continue to protect the battery and the EMS must remain within the last valid limits or stop requesting power.

The entity model, discovery mechanism, command permissions, and dashboard design are still open.

### 3.6. Companion application

**PLANNED**

The current in-tree `Software/bms_companion` is a legacy copy and is not the source for Home-BESS work. The planned `Software/Companion` is a FlexBMS-owned BMS-only application with two builds: a direct USB Web Serial build for independent service/development diagnostics, and an ESP32-hosted WebSocket build for local maintenance. Both provide live battery diagnostics and named service requests. Only direct USB has a raw development terminal; the network build has no raw terminal, exposes an explicit allowlist, and adds Gateway status and firmware-update workflow.

## 4. Operating concept

---

### 4.1. Start-up

At start-up, the STM32 validates its configuration and initializes the complete TPL chain. It assigns each battery monitor a chain address, configures its registers, runs diagnostics, and obtains a fresh measurement set. Both HV outputs remain off. After BMS validation, the separate HV supervisor can run its self-test and precharge sequence only when a new run request is supplied.

### 4.2. Normal operation

During normal operation the STM32 repeatedly measures the pack and evaluates its safety state. Battery limits and status are made available to the surrounding system. The inverter converts energy between the DC battery and the AC installation, while the future EMS decides the desired operating point.

### 4.3. Faulted operation

A battery, measurement, or communication fault immediately removes relay permission. Monitoring continues where possible so that the live condition can be observed. Faults remain latched until they are acknowledged through the supported service path and the BMS has successfully revalidated the battery chain.

The BMS exposes the HV supervisor as one aggregate blocking fault while the supervisor retains separate active, latched, and historical HV reasons. The Companion clear command can clear an HV latch only when the live HV conditions are healthy. The BMS then repeats chain initialization, diagnostics, and a complete fault-checked measurement cycle before the latch is released. If the retained run request is true, the BMS may then resume its normal startup sequence. Historical reasons remain available until reboot.

Configuration and initial TPL setup faults require a reboot. The ESP32, EMS, and Home Assistant cannot override a blocking STM32 fault.

## 5. Interfaces and ownership

| Information or action            | Owner          | Consumer                                |
|----------------------------------|----------------|-----------------------------------------|
| Cell voltages and temperatures   | STM32 BMS      | Diagnostics, inverter limits, telemetry |
| Battery current and SoC          | STM32 BMS      | Inverter, EMS, Home Assistant           |
| Battery safety limits            | STM32 BMS      | Inverter and EMS                        |
| Contactor and precharge          | STM32 BMS      | Physical HV path                        |
| Inverter operating state         | Inverter       | ESP32 / EMS / Home Assistant            |
| Requested charge/discharge power | EMS            | Inverter-facing control interface       |
| Dashboards and automation        | Home Assistant | User and household services             |

The protocol between these layers should carry explicit validity and freshness information. A stale value must never be interpreted as a current permission to charge, discharge, or close the contactor.

## 6. Current project structure

| Repository area        | Contents                                                                 |
|------------------------|--------------------------------------------------------------------------|
| Hardware/Master        | Master schematic, PCB, harnesses, and production data.                   |
| Hardware/ModuleBoard   | Battery module-board schematic, PCB, and production data.                |
| Software/Master        | STM32G491 firmware and platform configuration.                           |
| Software/bms_companion | Legacy in-tree Companion copy; not a Home-BESS source of truth.          |
| Software/Companion     | Planned FlexBMS-owned BMS maintenance UI with USB and Gateway builds.    |
| Software/Gateway       | Planned ESP32 network, MQTT, maintenance, OTA, and UART bridge firmware. |
| Software/protocol      | Planned STM32–ESP32 framing specification and test vectors.              |
| Simulations            | Electrical simulation files used during hardware development.            |
| Documentation          | This system overview and its Typst build setup.                          |

The repository does not yet contain ESP32, EMS, Home Assistant, or FlexBMS Companion implementation directories. The ESP32, Companion, and protocol layouts are agreed; their implementation remains planned.

## 7. Open topics

This overview is intentionally incomplete. The next useful documentation additions are:

- confirmed as-built module, NTC, and high-voltage topology;
- final inverter selection, protocol compatibility, and commissioning sequence;
- system-level operating, fault, recovery, and shutdown policy;
- commissioning of the ESP32 telemetry, local-network, and update architecture; and
- EMS and Home Assistant interfaces, fallback behavior, and user permissions.